



Department of *Computer Science and Engineering*
United International University

Software Testing and Quality Assurance

Group- 04 (Section B)

Group Name : 100G

Student Name	Student ID
Shekh Rifat Islam	011213036
Arshad Md. Adel	011221299
MD. Shoaib Siddiqi Shaon	011213164
Al Shahrier Khan Nipu	011202173
Jannatul Ferdous Swarna	011201157

Table of content

1	Test Plan.....	4
1.1	Check Health.....	4
1.2	Reset.....	4
1.3	Sign Up.....	4
1.4	Log In.....	5
1.5	Log Out.....	5
1.6	Auth.....	6
1.7	Get All Users.....	6
1.8	Get User By ID.....	7
1.9	Get User By Username.....	7
2.0	Update User.....	7
2.1	Block User.....	8
2.2	Unblock User.....	9
2.3	Get My Conversation.....	9
2.4	Get My Conversation By Id.....	9
2.5	Create Conversation.....	10
2.6	Add Members.....	10
2.7	Remove Members.....	11
2.8	Send Message.....	12
2.9	Get Messages of a Conversation.....	12
2	Test Cases.....	13
2.1	TC-01.....	13
2.2	TC-02.....	13
2.3	TC-03.....	14
2.4	TC-04.....	16
2.5	TC-05.....	17
2.6	TC-06.....	18
2.7	TC-07.....	19
2.8	TC-08.....	20
2.9	TC-09.....	21

2.10	TC-10.....	22
2.11	TC-11.....	24
2.12	TC-12.....	25
2.13	TC-13.....	26
2.14	TC-14.....	28
2.15	TC-15.....	30
2.16	TC-16.....	31
2.17	TC-17.....	31
2.18	TC-18.....	32
2.19	TC-19.....	33
2.20	TC-20.....	34
3	Defect Reports.....	36
	D-01: Null or Empty Username, Password, Email Accepted.....	36
	D-02: Unverified or Dummy Email Accepted.....	36
	D-03: Logout Allowed for Non-Logged-In or Non-Existent User.....	37
	D-04: Updating Email Without Verification.....	37
	D-05: Updating User with Empty Username, Email, Password.....	38
	D-06: User Can Block Self.....	38
	D-07: Creating Direct Conversation with Self.....	38
	D-08: Creating Direct Conversation with Multiple Users.....	39
	D-09: Creating Direct Conversation with Non-Existent User.....	39
	D-10: Creating Direct Conversation with Null or Empty Type.....	40
	D-11: Creating Group Conversation Without Group Name.....	40
	D-12: Creating Conversation with Blocked User.....	41
	D-13: Adding Non-Existent Member to Conversation.....	41
	D-14: Adding Blocked Member to Conversation.....	41
	D-15: Adding Multiple Members to Direct Conversation.....	42
4	Risk & Recommendations.....	42
	4.1 Broken Input Validation in login ,signup & Data Integrity.....	42
	4.2 Authentication & Authorization Bypass.....	43
	4.3 Identity Failure and Broken Conversation Model.....	43
5	Individual Reflections.....	44

1 Test Plan

1.1 Check Health

API endpoint : `http://localhost:8080/health`

Method : GET

Input Parameter : none

Test Case : TC-01

Description : This test ensures that the app server is up and running and able to respond. It ensures if the system is running or not before we run other tests. Here we have a very simple diagnostic endpoint which says service is available and running successfully.

1.2 Reset

API endpoint : `http://localhost:8080/reset`

Method : GET

Input Parameter : none

Test Case : TC-02

Description : This test restores all the data and puts the system back to its first stage. It's also used to provide a clean testing environment between test runs As a result previous test data does not affect with the current test results which ensures a reliable test environment.

1.3 Sign Up

API endpoint : `http://localhost:8080/auth/signup`

Method : POST

Input Parameter : `Bearer <JWT_TOKEN>, username, email, password`

Representative Values (token) : empty, invalid, valid

Representative Values (username) : null, present

Representative Values (email) : empty, invalid, valid

Representative Values (password) : null, present

Test Case : TC-03

Description : Sign Up verifies that new users can register to the System. It checks that functionality for the user registration is working fine with valid credentials. This ensures that the user onboarding is working as it is expected.

1.4 Log In

API endpoint : `http://localhost:8080/auth/login`

Method : POST

Input Parameter : `Bearer <JWT_TOKEN>, username, password`

Representative Values (token) : empty, invalid, valid

Representative Values (username) : null, existent, non-existent

Representative Values (password) : null, valid, invalid

Test Case : TC-04

Description : Log in verifies that registered members can authenticate and log in to the system. It confirms the mechanism for authenticating usernames and passwords works. This is necessary when access to the application must be authorized.

1.5 Log Out

API endpoint : `http://localhost:8080/auth/logout?username={username}`

Method : POST

Input Parameter : `username`

Representative Values (username) : null, existent, non-existent

Test Case : TC-05

Description : This test makes sure users can log out and leave system cleanly. It tests that the logout operation clears authentication tokens, and prevents unauthorized access after logging out. This is important for security and session handling.

1.6 Auth

API endpoint : `http://localhost:8080/auth/test`

Method : GET

Input Parameter : `Bearer <JWT_TOKEN>`

Representative Values (token) : empty, invalid, valid

Test Case : TC-06

Description : This test confirms that the authorization token or session is working. This guarantees that trust-protected endpoints perform user authentication checks before allowing access. It prevents unauthorized access to sensitive resources.

1.7 Get All Users

API endpoint : `http://localhost:8080/users/`

Method : GET

Input Parameter : `Bearer <JWT_TOKEN>`

Representative Values (token) : null, valid(logged in), invalid(logged out)

Test Case : TC-06

Description : In this test case, the test checks if we can get all users from the system. It makes sure that the user listing functionality is working fine, It gives us well formatted data. This is generally for administrative or user exploration type

of features.

1.8 Get User By ID

API endpoint : `http://localhost:8080/users/{id}`

Method : GET

Input Parameter : `Bearer <JWT_TOKEN>, id`

Representative Values (token) : null, valid(logged in), invalid(logged out)

Representative Values (id) : existent, non-existent

Test Case : TC-07

Description : This test ensures whether we can retrieve any profile by its id or not. This verifies that the system is able to pull in any particular user data successfully. It's very useful to check profiles, when taking actions on users.

1.9 Get User By Username

API endpoint : `http://localhost:8080/users/{username}`

Method : GET

Input Parameter : `Bearer <JWT_TOKEN>, username`

Representative Values (token) : null, valid(logged in), invalid(logged out)

Representative Values (username) : existent, non-existent

Test Case : TC-08

Description : This test ensures that it is possible to search for users based on their username. This helps username-based search work properly. This can be helpful in discovering different users without being aware of their ID.

2.0 Update User

API endpoint : `http://localhost:8080/users/{id}`

Method : PATCH

Input Parameter : `Bearer <JWT_TOKEN>,id, username, email, password`

Representative Values (id) : same-user, other-user

Representative Values (token) : null, valid(logged in), invalid(logged out)

Representative Values (username) : null, present

Representative Values (email) : null, present

Representative Values (password) : null, present

Test Case : TC-09

Description : This test confirms whether or not a user's profile information can be updated. It provides a way for users to update their name, email and other profile information. And that verifies the edit function works to retain data integrity.

2.1 Block User

API endpoint : `http://localhost:8080/users/block/{id}`

Method : PATCH

Input Parameter : `Bearer <JWT_TOKEN>,id`

Representative Values (id) : same-user, other-user

Representative Values (token) : null, valid(logged in), invalid(logged out)

Test Case : TC-10

Description : This test confirms that users can be blocked from the system at any cases like reduce destructions. Except it blocks the user from logging in or doing something. This also allows for moderation, and also security features. This aids in controlling misuse or violations of policy. Allowing to curate a more positive online environment.

2.2 Unblock User

API endpoint : `http://localhost:8080/users/block/{id}`

Method : DELETE

Input Parameter : `Bearer <JWT_TOKEN>,id`

Representative Values (id) : same-user, other-user

Representative Values (token) : null, valid(logged in), invalid(logged out)

Test Case : TC-11

Description : This test case makes sure that a blocked user can be restored communication, interaction and re-enter. It tests updates to status in the system. Additionally, It indicates that something you were not able to do is now available. This supports flexible user management.

2.3 Get My Conversation

API endpoint : `http://localhost:8080/conversations/get`

Method : GET

Input Parameter : `Bearer <JWT_TOKEN>`

Representative Values (token) : null, valid(logged in), invalid(logged out)

Test Case : TC-12

Description : This test asserts that a user can fetch all conversations they belong to. It enforces the proper access control as well as data filtering. It ensures that users can only view their own conversations. This supports conversation management, care and protection of nature.

2.4 Get My Conversation By Id

API endpoint : `http://localhost:8080/conversations/{conversationId}`

Method : GET

Input Parameter : `Bearer <JWT_TOKEN>, conversationId`

Representative Values (token) : null, valid(logged in), invalid(logged out)

Representative Values (conversationId) : user-in, user-out

Test Case : TC-13

Description : This test checks that a user can retrieve a specific conversation by its ID. It enforces that the authorization check is stopping people from accessing others' conversations, tracking, managing. Moreover, It also verifies the correct messages and member information was retrieved. This improves conversation accuracy.

2.5 Create Conversation

API endpoint : `http://localhost:8080/conversations/create`

Method : POST

Input Parameter : `Bearer <JWT_TOKEN>, type, name, memberIds`

Representative Values (token) : null, valid(logged in), invalid(logged out)

Representative Values (type) : null, DIRECT, GROUP

Representative Values (name) : null, not null

Representative Values (memberIds) : null, one, many

Test Case : TC-14

Description : This is a test that new conversations can be created by users. It tests for required fields, member assignment and conversation starting. It helps to store and show messages in the right place. Main purposes are people initiate and engage. It's an essential part of the communication experience.

2.6 Add Members

API endpoint : `http://localhost:8080/conversations/addMember`

Method : POST

Input Parameter : `Bearer <JWT_TOKEN>, conversationId, members`

Representative Values (token) : null, valid(logged in), invalid(logged out)

Representative Values (conversationId) : null, DIRECT, GROUP

Representative Values (members) : null, one, many, existent, non-existent, blocked, unblocked

Test Case : TC-15

Description : This test asserts that a new member can be added to an ongoing conversation, and collaboration, communication. It guarantees permissions are checked. It proves that new users have access properly. This is ideal for collaboration and growing a group.

2.7 Remove Members

API endpoint : `http://localhost:8080/conversations/removeMember`

Method : POST

Input Parameter : `Bearer <JWT_TOKEN>, conversationId, members`

Representative Values (token) : null, valid(logged in), invalid(logged out)

Representative Values (conversationId) : null, DIRECT, GROUP

Representative Values (members) : null, one, many, existent, non-existent, blocked, unblocked

Test Case : TC-15

Description : The purpose of the Remove Members test case is to verify that authorized users can successfully remove participants from a conversation or group. It ensures proper permission checks are enforced so only allowed users can remove members. This test also validates that removed users immediately lose access to the conversation and its messages. It helps maintain correct conversation membership, security, and access control.

2.8 Send Message

API endpoint : `http://localhost:8080/messages/{conversationId}/send`

Method : POST

Input Parameter : `Bearer <JWT_TOKEN>, conversationId, message`

Representative Values (token) : null, valid(logged in), invalid(logged out)

Representative Values (conversationId) : null, DIRECT, GROUP

Representative Values (message) : null, not null

Representative Values (member) : blocked, not blocked, existent, non-existent

Test Case : TC-16

Description : This test verifies that the user can post messages within a conversation. It verifies the creation, storage and share of messages in the right conversation. This is the basic feature of any messaging system.

2.9 Get Messages of a Conversation

API endpoint :

`http://localhost:8080/messages/{conversationId}/get?page=0&size=20`

Method : GET

Input Parameter : `Bearer <JWT_TOKEN>, conversationId`

Representative Values (token) : null, valid(logged in), invalid(logged out)

Representative Values (conversationId) : null, DIRECT, GROUP

Representative Values (member) : blocked, not blocked, existent, non-existent

Test Case : TC-17

Description : The test collects all the messages in a conversation. It confirms that

the messages history is preserved and can be fetched in a proper way. This means that users can see full conversation threads.

2 Test Cases

2.1 TC-01

ID	TC-01					
Short Title	Checking Server Status					
Pre-condition	Run the server using gradle on port 8080					
Description	Here we are checking if the server is responding correctly					
Scenario	Test Steps	Output	Expected	Status	Comment	Evidence
1. Server has been run now checking if servers responds	GET /health	200 OK	200 OK	PASS	success	https://drive.google.com/file/d/1J1tkNaJrSJIATF8rONcpB2OtT0qShOKz/view?usp=drive_link

2.2 TC-02

ID	TC-02
----	-------

Short Title	Checking if Server can reset					
Pre-condition	Run the server using gradle on port 8080 and check health					
Description	Here we are checking if the server can be reset					
Scenario	Test Steps	Output	Expected	Status	Comment	Evidence
1. Server has been run now checking if it can reset	GET /reset	200 OK	200 OK	PASS	success	https://drive.google.com/file/d/1QuUxETM9N7xIMglfAd1UHgsoE2ur4bl/view?usp=drive_link

2.3 TC-03

ID	TC-03					
Short Title	Checking if a user can sign up					
Pre-condition	Run the server using gradle on port 8080					
Description	Here we are checking if a user can sign up to the messenger					
Scenario	Test Steps	Output	Expected	Status	Comment	Evidence
1. Giving valid username, email, password as input to check if it works correctly	POST/auth/signup username: Rifat email: alice@test.com password: 1234	201 Created	201 Created	PASS	Success	https://drive.google.com/file/d/1uCZSPD2bxE6kU_n58npgWbleevuRj1hx/view?usp=drive_link
2. Sign up using the same user	POST/auth/signup username: Rifat email: alice@test.com password: 1234	400 Bad Request	400 Bad Request	PASS	Success	https://drive.google.com/file/d/13PyDV_zpsZOJOSA2MV_FliMNYGbx7VzL/view?usp=drive_link

3. Giving username null	POST/auth/signup username: null email: alice@test.com password: 1234	500 Internal Server Error	400 Bad Request	FAIL	Defect	https://drive.google.com/file/d/1blZkg_UKY9wwz33On-QDpLPDF1JiifOs/view?usp=drive_link
4. Giving username as empty string	POST/auth/signup username: "" email: alice@test.com password: 1234	201 Created	400 Bad Request	FAIL	Defect	https://drive.google.com/file/d/10xDRaUnFnmEtxEggJTt0BLflvbjdggFm/view?usp=drive_link
5. Giving password null	POST/auth/signup username: Rifatt email: alice@test.com password: null	201 Created	400 Bad Request	FAIL	Defect	https://drive.google.com/file/d/18X9-ESrLSn4KQ35EnwD83906vHV6nof3/view?usp=drive_link
6. Giving password empty	POST/auth/signup username: Rifattt email: alice@test.com password:	201 Created	400 Bad Request	FAIL	Defect	https://drive.google.com/file/d/1NfEOWmeO2p9ks_UIVKLIFROa6Ela8bSB/view?usp=drive_link
7. Giving email null	POST/auth/signup username: Rit Email: null Password: 1234	201 Created	400 Bad Request	FAIL	Defect	https://drive.google.com/file/d/1qc1sX2O2Fa1QIHQ8tCIZyuLkUXUXL3K/view?usp=drive_link

8. Giving email empty	POST/auth/signup username: Ritt Email: Password: 1234	201 Created	400 Bad Request	FAIL	Defect	https://drive.google.com/file/d/1bLmtKtQqKMAlg0LTUYF1McE4Peun7ghl/view?usp=drive_link
9. Giving unverified or dummy email	POST/auth/signup username: Adel Email: dummy@gmail.com Password: 1234	201 Created	403 Forbidden	FAIL	Defect	https://drive.google.com/file/d/1o9erT2YI_togSABtpnL-i5TV8vHbdMkA/view?usp=drive_link

2.4 TC-04

ID	TC-04					
Short Title	Checking if a user can log in after sign up					
Pre-condition	Server should be running					
Description	Checking if a user can log in to the messenger					
Scenario	Test Steps	Output	Expected	Status	Comment	Evidence
1. Log in using existing user with valid pass, email	POST/auth/login username: Rifat password: 1234	200 OK + Token	200 OK + Token	PASS	Success	https://drive.google.com/file/d/1IV7e5mLdA3fSzI8FnCMsuW_Pt7Uk9rJ/view?usp=drive_link
2. Log in using incorrect password	POST/auth/login username: Rifat password: 12345	401 Unauthorized Invalid	401 Unauthoriz ed	PASS	Success	https://drive.google.com/file/d/1WtAatBIRT-g2

		password				98FUTCO QP2Tj4kXa nsQ_/view? usp=drive_ link
3. Log in with non existing user	POST/auth/login username: Someone password: abcd	401 Unauthorized User not found	401 Unauthoriz ed	PASS	Success	https://drive .google.co m/file/d/19s pAB2AuRA f66GdpssS etg8aOhWi cQz4/view? usp=drive_ link

2.5 TC-05

ID	TC-05					
Short Title	Checking if a user can log out after log in					
Pre-condition	User should be logged in					
Description	Checking if a user can log out to the messenger					
Scenario	Test Steps	Output	Expected	Status	Comment	Evidence
1. Logging out logged in user	POST /auth/logout?username={username} username: Rifat	200 OK + User logged out successfully	200 OK	PASS	Success	https://drive .google.co m/file/d/1a0 y1vyMemF UXGlt6lItD nICr4_gTau 4V/view?us p=drive_ link
2. Logging out using any sign up user (not logged in now)	POST /auth/logout?username={username} username: Adel	200 OK + User logged out successfully	401 Unauthoriz ed	FAIL	Defect	https://drive .google.co m/file/d/1la Wo7CMID mFb49W3q jypfwbrr4h AgwK/view ?usp=drive _link

3. Logging out using Non existing user	POST /auth/logout?username={username} username: anyuser	200 OK + User logged out successfully	401 Unauthorized	FAIL	Defect	https://drive.google.com/file/d/1AAlilEW_iRPFtJhIQpznTtMw4uULD8Oz/view?usp=drive_link
--	---	--	---------------------	------	--------	---

2.6 TC-06

ID	TC-06					
Short Title	Checking if authentication token is valid					
Pre-condition	User must log in to get a token					
Description	Checking if the token works perfectly					
Scenario	Test Steps	Output	Expected	Status	Comment	Evidence
1. Checking if token works for logged in user	GET /auth/test Token : Bearer <JWT_TOKEN>	200 OK + Hello, Rifat! This is a protected API.	200 OK	PASS	Success	https://drive.google.com/file/d/1AUdTt2GBYrYFdJyQmEZ8opfgz_ge3zCK/view?usp=drive_link
2. Testing with a wrong token	GET /auth/test Token : wrong token	403 Forbidden	403 Forbidden	PASS	Success	https://drive.google.com/file/d/1ndJA_emp_z6FJR7eQXi0DPVg3fNqN4W5/view?usp=drive_link

3. Testing with no token at all	GET /auth/test Token : no token	403 Forbidden	403 Forbidden	PASS	Success	https://drive.google.com/file/d/1tzjJWUj4nmK1Ph7dUBMSdLWJp0dpaWH2/view?usp=drive_link
---------------------------------	--	---------------	---------------	------	---------	---

2.7 TC-07

ID	TC-07					
Short Title	Getting All Users					
Pre-condition	User must log in to get a token					
Description	Purpose is to identify if we can fetch all the users					
Scenario	Test Steps	Output	Expected	Status	Comment	Evidence
1. Hitting the API to get all users using valid token	GET /users/ Token : Bearer <JWT_TOKEN>	200 OK + JSON response of all the users	200 OK	PASS	Success	https://drive.google.com/file/d/15rdJ2fMAhUWSp8NS6ladREO-dYeGntuL/view?usp=drive_link
2. Trying to fetch using wrong token	GET /auth/test Token : wrong token	401 Unauthorized	401 Unauthorized	PASS	Success	https://drive.google.com/file/d/18xnc1C93QpOdm2yAWlOwojNf1ByzG3o/view?usp=drive_link

3. Fetching with no token at all	GET /auth/test Token : no token	401 Unauthorized	401 Unauthoriz ed	PASS	Success	https://drive.google.com/file/d/1stgvfp9ICxA7xM0rMnUzpeHdgwXjSsGp/view?usp=drive_link
----------------------------------	--	---------------------	-------------------------	------	---------	---

2.8 TC-08

ID	TC-08					
Short Title	Getting a single User by id					
Pre-condition	User must log in to get a token					
Description	Purpose is to identify if we can fetch all the users					
Scenario	Test Steps	Output	Expected	Status	Comment	Evidence
1. Trying to get details of a existent user	GET /users/{id} Token : Bearer <JWT_TOKEN> Id : existing user	200 OK + JSON response of all the users	200 OK	PASS	Success	https://drive.google.com/file/d/1E6pqctOD4ciNlxN_p34yfzroyqzsZMEi/view?usp=drive_link
2. Trying to fetch a non existent user	GET /users/{id} Token : Bearer <JWT_TOKEN> Id : non-existing user	404 Not Found + User Not Found	404 Not Found	PASS	Success	https://drive.google.com/file/d/1QPGCKjCzSLb_JP2oeqbZOiE5JoMFZ4U/view?usp=drive_link
3. Trying to fetch using wrong token	GET /users/{id} Token : wrong token	401 Unauthorized	401 Unauthoriz ed	PASS	Success	https://drive.google.com/file/d/1nL

						HWRB8ncGN6NDnW4g00eS_n32uUpP2T/view?usp=drive_link
4. Fetching with no token at all	GET /users/{id} Token : no token	401 Unauthorized	401 Unauthorized	PASS	Success	https://drive.google.com/file/d/1304hAoD19x7ENhb2CxOHhDOhjSxaQEvA/view?usp=drive_link

2.9 TC-09

ID	TC-09					
Short Title	Getting a single Users using username					
Pre-condition	User must log in to get a token					
Description	Purpose is to identify if we can fetch all the users					
Scenario	Test Steps	Output	Expected	Status	Comment	Evidence
1. Trying to get details of a existent user	GET /users/{username} Token : Bearer <JWT_TOKEN> Id : existing user	200 OK + JSON response of all the users	200 OK	PASS	Success	https://drive.google.com/file/d/157bqmtQOYbjwPYyyMMVdKi5Gab777nG1/view?usp=drive_link
2. Fetching a non existing user	GET /users/{id} Token : Bearer <JWT_TOKEN> Id : non-existing	404 Not Found + User Not Found	404 Not Found	PASS	Success	https://drive.google.com/file/d/1VEcUILxgzKqLOw8bvp

	user					XhUss2HwBvDfp/view?usp=drive_link
3. Trying to fetch using wrong token	GET /users/{id} Token : wrong token	401 Unauthorized	401 Unauthorized	PASS	Success	https://drive.google.com/file/d/10RW6HjPaLv4i3grZu3lAVjnNpmbQcOD/view?usp=drive_link
4. Fetching with no token at all	GET /users/{id} Token : no token	401 Unauthorized	401 Unauthorized	PASS	Success	https://drive.google.com/file/d/1mv9Ss1Uu7LcSXqcdshinMFouRVnZliMY/view?usp=drive_link

2.10 TC-10

ID	TC-10					
Short Title	Update user details					
Pre-condition	User must be created to update					
Description	Purpose is to identify if we can fetch all the users					
Scenario	Test Steps	Output	Expected	Status	Comment	Evidence

1. Updating username and password	PATCH /users/{id} Id : 101 username : Rifatt Password : new_pass	200 OK + JSON response	200 OK	PASS	Success	https://drive.google.com/file/d/1QesJn1PgMO-RDkgnaQf7UuCjA0BqSukt/view?usp=drive_link
2. Updating email	PATCH /users/{id} Id : 101 Email: demo@gmail.com Password : new_pass	200 OK + JSON response	403 Forbidden	FAIL	Defect	https://drive.google.com/file/d/1QKSJ-jBwGLEuW7rZSAFwhiTgxGGF9rrN/view?usp=drive_link
3. Updating other user	PATCH /users/{id} Id : 102 username : Rifattt Password : new_pass	403 Forbidden + You can only update own details	403 Forbidden	PASS	Success	https://drive.google.com/file/d/1Agu7Od7ICKgNE1ivyjN-X6KRXyJ3GN8w/view?usp=drive_link
4. Update non existing user on server	PATCH /users/{id} Id : 1055 username : Rifattt Password : new_pass	400 Bad Request + User not found	400 Bad Request	PASS	Success	https://drive.google.com/file/d/1mHJdeE7a2kluH8to0W6FbHChR6z8kYbW/view?usp=drive_link
5. Updating user with empty username, email, password	PATCH /users/{id} Id : 109 username : "" Email : "" Password : ""	200 OK + JSON response	400 Bad Request	FAIL	Defect	https://drive.google.com/file/d/15Th2YF39NBX1BZq3-iS2G5AZ7XnaJWW0/view?usp=drive_link

2.11 TC-11

ID	TC-11					
Short Title	Block user					
Pre-condition	Log in to the system					
Description	Checking if the block function works properly					
Scenario	Test Steps	Output	Expected	Status	Comment	Evidence
1. Blocking self	PATCH /users/block/{id} Id : 101 (self)	200 OK + JSON response	403 Forbidden	Fail	Defect	https://drive.google.com/file/d/1opwNB79lvxQHCSbjJKkiZns2O6tTBvLD/view?usp=drive_link
2. Blocking other	PATCH /users/block/{id} Id : 102	200 OK + JSON response	200 OK	PASS	Success	https://drive.google.com/file/d/1vq18zdUAv0rhBXj-jDc7v0bUS8Ym_G3/view?usp=drive_link
3. Blocking non existent user	PATCH /users/block/{id} Id : 107	400 Bad Request + User to be blocked not found	400 Bad Request	PASS	Success	https://drive.google.com/file/d/1HXl4HjKW3kfwXQlgWPI-BkklLPfoyyhB/view?usp=drive_link

4. Blocking blocked user again	PATCH /users/block/{id} Id : 102	200 OK + JSON response	400 Bad Request	PASS	Success	https://drive.google.com/file/d/1uOeu4qKScHEu-W1MRbKA9QIXCLRPFG3a/view?usp=drive_link
5. Blocking without auth token	PATCH /users/block/{id} Token : none	401 Unauthorized + No auth token provided	401 Unauthoriz ed	PASS	Success	https://drive.google.com/file/d/1Q7dxH0hnlZFMLLtU7cbUOP3OFw6_Xad/view?usp=drive_link

2.12 TC-12

ID	TC-12					
Short Title	Unblock user					
Pre-condition	Log in to the system					
Description	Checking if the unblock system works perfectly					
Scenario	Test Steps	Output	Expected	Status	Comment	Evidence
1. Unblocking self	DELETE /users/block/{id} Id : 101 (self)	200 OK + JSON response	400 Bad Request	FAIL	Defect	https://drive.google.com/file/d/1Oo3-s9-g3p1slweCTLgRnhZjtXUaS_zOu/view?usp=drive_link

2. Unblocking non existent user	DELETE /users/block/{id} Id : 107	400 Bad Request + User to be unblocked to found	400 Bad Request	PASS	Success	https://drive.google.com/file/d/1xaZHNYcr6UqqPW23uS-akQ0u08Ow4MHo/view?usp=drive_link
3. Unblocking blocked user	DELETE /users/block/{id} Id : 102	200 OK + JSON response	200 OK	PASS	Success	https://drive.google.com/file/d/19pHmvDCIfBcEV5Ptuz3qOEuAY3mUuy/view?usp=drive_link
4. Unblocking unblocked user again	DELETE /users/block/{id} Id : 102	400 Bad Request + Failed to unlock user	400 Bad Request	PASS	Success	https://drive.google.com/file/d/1_P19uItrBBebt05mgVRrvLONRNUGzljw/view?usp=drive_link
5. Unblocking without auth token	DELETE /users/block/{id} Token : none	401 Unauthorized + No auth token provided	401 Unauthoriz ed	PASS	Success	https://drive.google.com/file/d/1XGop8CfVYytaLmxSKr9HR8SkGUm7IPCA/view?usp=drive_link

2.13 TC-13

ID	TC-13
Short Title	Create DIRECT conversation

Pre-condition	Must be logged and valid user					
Description	Checking if we could open DIRECT conversation					
Scenario	Test Steps	Output	Expected	Status	Comment	Evidence
1. Creating direct conversation with other user	POST /conversations/create Id : 102	200 OK + JSON response	200 OK	PASS	Success	https://drive.google.com/file/d/1m5ggrACdXXsvRdbDKdFWwypCaHKEOt6Y/view?usp=drive_link
2. Creating direct conversation with self	POST /conversations/create Id : 101 (self)	200 OK + JSON response	400 Bad Request	FAIL	Defect	https://drive.google.com/file/d/1v4-mTd5q210L4eEz3jKHrY2hQZ_8vYin/view?usp=drive_link
3. Creating direct conversation with multiple users	POST /conversations/create Id : 103,105,107	200 OK + JSON response	400 Bad Request	FAIL	Defect	https://drive.google.com/file/d/1gTrhzEd4oArl9asyZRI1qgR6Vok3vQW6/view?usp=drive_link
4. Creating direct conversation with non-existent user	POST /conversations/create Id : 10111	200 OK + JSON response	400 Bad Request	FAIL	Defect	https://drive.google.com/file/d/156wDzuTgH9efESFyyfOE-OVQfOq2SSa0/view?usp=drive_link
5. Creating direct conversation with type = null/empty	POST /conversations/create Type : null / ""	200 OK + JSON response	400 Bad Request	FAIL	Defect	https://drive.google.com/file/d/1-wTvqHMXxH8xH6iB2lciJ

						046ubm9SML9/view?usp=drive_link
6. Creating direct conversation without a type	POST /conversations/create Type : ""	200 OK + JSON response	400 Bad Request	FAIL	Defect	https://drive.google.com/file/d/1a_wfNnvJz70e-PM8pATgvtTCKKB7nXw_E/view?usp=drive_link
7. Creating direct conversation with a blocked user	POST /conversations/create Id : 103 (blocked user)	200 OK + JSON response	400 Bad Request	FAIL	Defect	https://drive.google.com/file/d/1NJZbRQqFhltwxuMXnWbVnoAi6vc_R_RB/view?usp=drive_link

2.14 TC-14

ID	TC-14
Short Title	Create GROUP conversation
Pre-condition	Must be logged and valid user

Description		Checking if we could open GROUP conversation				
Scenario	Test Steps	Output	Expected	Status	Comment	Evidence
1. Creating group conversation with other user	POST /conversations/create Id : 102	200 OK + JSON response	200 OK	PASS	Success	https://drive.google.com/file/d/1U4io-LmSboMrEMgdvfgmmT94nf3lQu2l/view?usp=drive_link
2. Creating group conversation with self	POST /conversations/create Id : 101 (self)	200 OK + JSON response	400 Bad Request	FAIL	Defect	https://drive.google.com/file/d/1kV6MnuEuWVtK4xiYZcB3SrN8HTsYCl6r/view?usp=drive_link
3. Creating group conversation with non-existent user	POST /conversations/create Id : 10777	200 OK + JSON response	400 Bad Request	FAIL	Defect	https://drive.google.com/file/d/1wkKZgUAvgGTLBxwHnbXylDSMbRLQQSM2/view?usp=drive_link
4. Creating group conversation with type = null/empty	POST /conversations/create Type : null / ""	200 OK + JSON response	400 Bad Request	FAIL	Defect	https://drive.google.com/file/d/13ykAahj4jnsyjmBRUn3bGhNsRBoZ1hwu/view?usp=drive_link
5. Creating group conversation without a group name	POST /conversations/create Name : null/ ""	200 OK + JSON response	400 Bad Request	FAIL	Defect	https://drive.google.com/file/d/1DJOFigEKNm0Gmi_T9WKkYAX8OA

						CioWSg/view?usp=drive_link
6. Creating group conversation with a blocked user	POST /conversations/create Id : 103 (blocked user)	200 OK + JSON response	400 Bad Request	FAIL	Defect	https://drive.google.com/file/d/1TAy8Z-RBSADhc1hKTIDjuKYiaEpLh5XH/view?usp=drive_link

2.15 TC-15

ID	TC-15					
Short Title	Get all conversation					
Pre-condition	Must be logged and valid user					
Description	Test purpose to see if server can fetch all conversations user enrolled in					
Scenario	Test Steps	Output	Expected	Status	Comment	Evidence
1. Fetching with auth	GET /conversations/get Auth : token	200 OK + JSON response	200 OK	PASS	Success	https://drive.google.com/file/d/1_vEUxjBpgu2ORPdETbj9kRjazOEv6s92/view?usp=drive_link
2. Fetching without auth	GET /conversations/get Auth : none/false	500 Internal Server Not Found	401 Missing/Invalid Auth	FAIL	Defect	https://drive.google.com/file/d/1QPX9bs36QXSOaEwutGQ2KMqkxwX4DFyn/view?usp=drive_link

2.16 TC-16

ID	TC-16					
Short Title	Get any conversation by id					
Pre-condition	Must be logged and valid user					
Description	This test is for checking whether the server can fetch a single conversation by their id					
Scenario	Test Steps	Output	Expected	Status	Comment	Evidence
1. Fetching with auth	GET /conversations/{conversationId} Auth : token	200 OK + JSON response	200 OK	PASS	Success	https://drive.google.com/file/d/1B-5VZvWM7CV_tGZ3jDQVVb45xj-yveXa/view?usp=drive_link
2. Fetching without auth	GET /conversations/{conversationId} Auth : none/false	500 Internal Server Not Found	401 Missing/Invalid Auth	FAIL	Defect	https://drive.google.com/file/d/19T5ef5mcY1Pzreg3_OjYsfDgsUA3YH/view?usp=drive_link
3. Trying to fetch a conversation that do not exist	GET /conversations/{conversationId} conversationId : non existing	400 Conversation not found	400 Conversation not found	PASS	Success	https://drive.google.com/file/d/1GGEjOT_582B9IFKVNQ4DZqn7_7NMAcgB/view?usp=drive_link

2.17 TC-17

ID	TC-17					
Short Title	Add members to direct conversation					
Pre-condition	Must be logged and valid user					
Description	The purpose of this testing is to adding members to the direct conversation					
Scenario	Test Steps	Output	Expected	Status	Comment	Evidence
1. Adding a signed up member	POST /conversations/addMember Members : 102	200 OK + JSON response	200 OK	PASS	Success	https://drive.google.com/file/d/1EwFMue2b3nNsAPohtI61CPRzFzXZDOUsS/view?usp=drive_link
2. Adding a non existent member	POST /conversations/addMember Members : 1077	200 OK + JSON response	403 Forbidden	FAIL	Defect	https://drive.google.com/file/d/1tz0ZRDoRo2DxLAovEP-3dxzMOGhghUn1/view?usp=drive_link
3. Adding a blocked member	POST /conversations/addMember Members : 103	200 OK + JSON response	403 Forbidden	FAIL	Defect	https://drive.google.com/file/d/1f56ripLzr8EQGcjWwdlzWNziizg2t04C/view?usp=drive_link

2.18 TC-18

ID	TC-18					
Short Title	Remove member					
Pre-condition	Must be logged and valid user					
Description	The purpose of this testing is to adding members to the GROUP conversation					
Scenario	Test Steps	Output	Expected	Status	Comment	Evidence
1. Remove a valid member from the conversation	POST /conversations/remove Member Members : 103	200 OK + JSON response	200 OK	PASS	Success	https://drive.google.com/file/d/1Cfgihn98kFrVoe8NoolAeKEQfapCGtLk/view?usp=drive_link
2. Removing a member that is not in that conversation	POST /conversations/remove Member Members : 104	200 OK + JSON response	403 Forbidden	FAIL	Defect	https://drive.google.com/file/d/1eSlnPQFXZDmfi3Cu09gx4wpoU46zTx5O/view?usp=drive_link

2.19 TC-19

ID	TC-19					
Short Title	Send message					
Pre-condition	Must be logged and valid user					
Description	Checking if the messaging functionality works nor not					
Scenario	Test Steps	Output	Expected	Status	Comment	Evidence
1. Sending message to a conversation that does not exist	POST /messages/{conversati	400 Bad Request +	400 Bad Request	PASS	Success	https://drive.google.com/file/d/1k

	onId}/send conversationId: 20888	Conversation not found				m6bx5xcXiOrHNr4o0PuDqV3TS7XZOJH/view?usp=drive_link
2. Sending message to a valid conversation	POST /messages/{conversationId}/send conversationId: 201	200 OK + JSON response	200 OK	PASS	Success	https://drive.google.com/file/d/1CylfhfViZelzUE01Ce-njHbXP7uY7pNS/view?usp=sharing
3. Sending a null/empty message	POST /messages/{conversationId}/send conversationId: 201 Body : null	400 Bad Request	400 Bad Request	PASS	Success	https://drive.google.com/file/d/17Ani085SO-HByK0ToZtr6h2CRXHLs-9b/view?usp=drive_link
4. Sending message to a blocked user	POST /messages/{conversationId}/send conversationId: 215	400 Bad Request + You must unblock the user to send message	400 Bad Request	PASS	Success	https://drive.google.com/file/d/1D9BkL0UqbSkuaesRzCGjta2miNL7rLK/view?usp=sharing
5. Sending message to a group conversation where blocked user also present there	POST /messages/{conversationId}/send conversationId: 202	200 OK + JSON response	200 OK	PASS	Success	https://drive.google.com/file/d/1vYQ-VRleAw6lBuw5Vf1LcEy6GGM1kG-K/view?usp=sharing

ID	TC-20					
Short Title	Get messages					
Pre-condition	Must be logged and valid user					
Description	After sending the message we have to check if we could also get the messages also					
Scenario	Test Steps	Output	Expected	Status	Comment	Evidence
1. Trying to fetch message without auth	GET /messages/{conversationId}/get?page=0&size=20 Token : none	500 Internal Server Error	400 Bad Request	FAIL	Defect	https://drive.google.com/file/d/1eahhT6_9pjDWRuZ2MMebTDmVGdw7x2ly/view?usp=drive_link
2. Trying to fetch message with valid token	GET /messages/{conversationId}/get?page=0&size=20 Token : Valid token	200 OK + JSON response	200 OK	PASS	Success	https://drive.google.com/file/d/1rgur2x4zy0rE8y5FbvSklo9bRfMNA8fI/view?usp=drive_link
3. Trying to fetch message of a conversation that does not exist	GET /messages/{conversationId}/get?page=0&size=20 conversationId : 2007	400 Bad Request + Conversation not found	400 Bad Request	PASS	Success	https://drive.google.com/file/d/12W19OaZJSLE8GhV858_0S8rrt5ykOCzD/view?usp=drive_link

3 Defect Reports

D-01: Null or Empty Username, Password, Email Accepted

Defect ID: D-01

Title: System allows null or empty ("") username, password, or email

Severity: Critical

Steps to Reproduce:

1. Open the sign-up API / form
2. Provide null or empty string ("") for username, password, and/or email
3. Submit the request

Expected Result:

The system should reject the request with proper validation errors (e.g., 400 Bad Request).

Actual Result:

The system accepts null or empty values and creates a user account.

Related Test Case: TC-05

D-02: Unverified or Dummy Email Accepted

Defect ID: D-02

Title: User can register using unverified or dummy email

Severity: High

Steps to Reproduce:

1. Sign up using a fake or non-existent email ()
2. Complete registration

Expected Result:

The system should validate email authenticity or restrict access until authentic email verification.

Actual Result:

User account is created and usable without authentic email verification.

Related Test Case: TC-04

D-03: Logout Allowed for Non-Logged-In or Non-Existent User

Defect ID: D-03

Title: Logout API works for users who are not logged in or do not exist

Severity: Medium

Steps to Reproduce:

1. Call logout API without logging in
2. Call logout API using a non-existent user ID

Expected Result:

The system should return an authentication error (401 Unauthorized).

Actual Result:

Logout API returns success response.

Related Test Case: TC-02

D-04: Updating Email Without Verification

Defect ID: D-04

Title: User can update email without verification

Severity: High

Steps to Reproduce:

1. Login as a valid user
2. Update email to a new address
3. Save changes

Expected Result:

Email update should require verification before being applied.

Actual Result:

Email is updated immediately without verification.

Related Test Case: TC-04

D-05: Updating User with Empty Username, Email, Password

Defect ID: D-05

Title: User profile update allows empty username, email, and password

Severity: Critical

Steps to Reproduce:

1. Login as a user
2. Update profile with empty values
3. Submit update

Expected Result:

Validation error should be returned.

Actual Result:

Profile updates successfully with invalid data.

Related Test Case: TC-05

D-06: User Can Block Self

Defect ID: D-06

Title: User can block their own account

Severity: Medium

Steps to Reproduce:

1. Login as a user
2. Use block API with own user ID

Expected Result:

System should prevent blocking himself.

Actual Result:

User successfully blocks their own account.

Related Test Case: TC-09

D-07: Creating Direct Conversation with Self

Defect ID: D-07

Title: Direct conversation can be created with self

Severity: Low

Steps to Reproduce:

1. Login as user
2. Create direct conversation himself using own user ID

Expected Result:

System should prevent self-conversation creation.

Actual Result:

Conversation is created successfully.

Related Test Case: TC-11

D-08: Creating Direct Conversation with Multiple Users

Defect ID: D-08

Title: Direct conversation allows multiple users

Severity: Medium

Steps to Reproduce:

1. Login
2. Create a direct conversation with more than one participant

Expected Result:

Direct conversation should allow only two users.

Actual Result:

Multiple users are added to a direct conversation.

Related Test Case: TC-15

D-09: Creating Direct Conversation with Non-Existent User

Defect ID: D-09

Title: Direct conversation created with non-existent user

Severity: High

Steps to Reproduce:

1. Login
2. Create direct conversation using invalid user ID

Expected Result:

System should reject the request.

Actual Result:

Conversation is created successfully.

Related Test Case: TC-09

D-10: Creating Direct Conversation with Null or Empty Type

Defect ID: D-10

Title: Conversation type can be null or empty

Severity: Medium

Steps to Reproduce:

1. Create conversation
2. Set `type = null` or `""`

Expected Result:

The system should validate the conversation type.

Actual Result:

Conversation is created without type.

Related Test Case: TC-10

D-11: Creating Group Conversation Without Group Name

Defect ID: D-11

Title: Group conversation created without a group name

Severity: Medium

Steps to Reproduce:

1. Log in
2. Do not provide group name
3. Do not provide group name

Expected Result:

Group name should be mandatory.

Actual Result:

The group is created without a name.

Related Test Case: TC-11

D-12: Creating Conversation with Blocked User

Defect ID: D-12

Title: Conversation allowed with blocked user

Severity: High

Steps to Reproduce:

1. Block a user Id
2. Create conversation with blocked user

Expected Result:

Conversation should not be allowed.

Actual Result:

Conversation is created successfully.

Related Test Case: TC-12

D-13: Adding Non-Existent Member to Conversation

Defect ID: D-13

Title: Non-existent user can be added as conversation member

Severity: High

Steps to Reproduce:

1. Create conversation
2. Add invalid user ID

Expected Result:

The system should reject invalid members.

Actual Result:

Members are added successfully.

Related Test Case: TC-13

D-14: Adding Blocked Member to Conversation

Defect ID: D-14

Title: user can be added a blocked user to group conversation

Severity: High

Steps to Reproduce:

1. Block a user
2. Add that user to in group conversation

Expected Result:

Blocked users should not be addable.

Actual Result:

The blocked user is added successfully in group conversation by myself.

Related Test Case: TC-14

D-15: Adding Multiple Members to Direct Conversation

Defect ID: D-15

Title: Direct conversation allows adding multiple members

Severity: Medium

Steps to Reproduce:

1. Create direct conversation
2. Add more than one member

Expected Result:

Direct conversation should allow only two users.

Actual Result:

Multiple users are added.

Related Test Case: TC-15

4 Risk & Recommendations

4.1 Broken Input Validation in login ,signup & Data Integrity

Related Defects: D-01, D-05, D-10, D-11

Description:

This system accepts null, empty, or invalid core fields in username, password, email in sign up process and in update process. Besides that in conversation process that also accept empty conversation type and group name.

Risk:

Defective access : Here it can face authorize user access and can easily modify any user information by some other or himself without permission. Besides that null value can give an unauthorized user access to the system.

Difficult data recovery : If this null value is not handled then it will be difficult to recover the user data also all the conversation.

Recommendations:

- Here first I need to handle null value for every input.
- Then we need to do verification for any kind of user information modification.
- Implement an error message to show those errors.

4.2 Authentication & Authorization Bypass

Description : Here users can create accounts without verifying their email, and log out without authentication. Besides they can change sensitive information like email without verification.

Risk : These issues can lead to account hacking, fake or spam accounts. Also, it can lead to violations of security standards and regulations.

Recommendation:

- Force proper authentication for every sensitive action.
- Users must verify their email before using the account.
- Any sensitive update must require re-verification.

4.3 Identity Failure and Broken Conversation Model

Description:

Users can block themselves. From the user side blocked users can still be contacted by each other and can be added to any group conversation. Direct chats do not have proper checks, so

the wrong number of users can be added, users can chat with themselves, and some users IDs are not valid.

Impact:

- Users could face harassment by the blocked user though he blocked the user.
- For this inconsistent design user trust will be broken and system workflow will hamper.
- Broken conversation models lead to logical errors in conversations, messages being sent to the wrong or non-existent users, and an overall confusing and broken user experience.

Recommendation :

- Need to identify the block id properly by username or user id.\
- A direct chat must always have exactly two different valid users and self-chat must be blocked completely.

5 Individual Reflections

Shekh Rifat Islam : I have tested multiple test cases. And found defects on signup and log out. There was no defect on log in. In signup a user can create a profile without username, email, password. They can also create profiles using null values. This is surely a defect. And also the email verification process is not good. We can register by any dummy email.

On the log out part when I hit API with other members id then the server also logs out them. Which is a big defect.

Arshad Md. Adel: Here I go through in this testing i work at fast case where i found some defect in **update user ,block user, unblock user** and **direct conversation** which i have mention in the test case table.Besides that figure out the defects explain that and risk that could held from the that defect scenario and suggest the mitigation.

MD. Shoaib Siddiqi Shaon: I have found defects in creating DIRECT/GROUP conversation. I can create these conversations without name,type. They also work with null as well which is a defect in real world scenarios.

Another big concern is I can create group with blocked members. Also conversations can be created with blocked members. These are the defects I found.

Al Shahrier Khan Nipu: I tried to test some cases like **TC- 7,8,16,17** and found some defects in **TC- 16,17**. But I couldn't find any defect in **TC- 7,8**. In **TC- 16** i've found defects while Fetching without auth and the output was 500 Internal Server Not Found, instead of 401 Missing/Invalid Auth. On the otherhand, In **TC-17** i've found defects while Adding a non existent member & Adding a blocked member.

Jannatul Ferdous Swarna: In **scenario 2** of **TC- 18**, I tried to remove a member who is not part of the conversation, and got returned 200 Ok instead of 403 Forbidden which is a defect I've found. And In **scenario 1** of **TC- 20**, I was trying to fetch a message without auth but instead of 400 Bad requests I've got 500 Internal Server Error which is also a defect. Getting internal server errors in 500 range instead of 400 range can hamper the execution of the process/server.